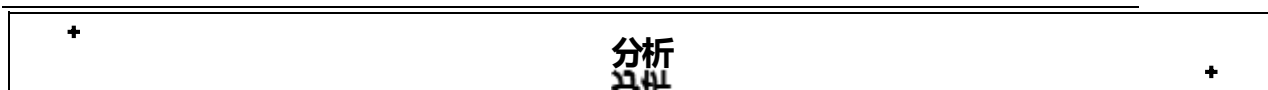




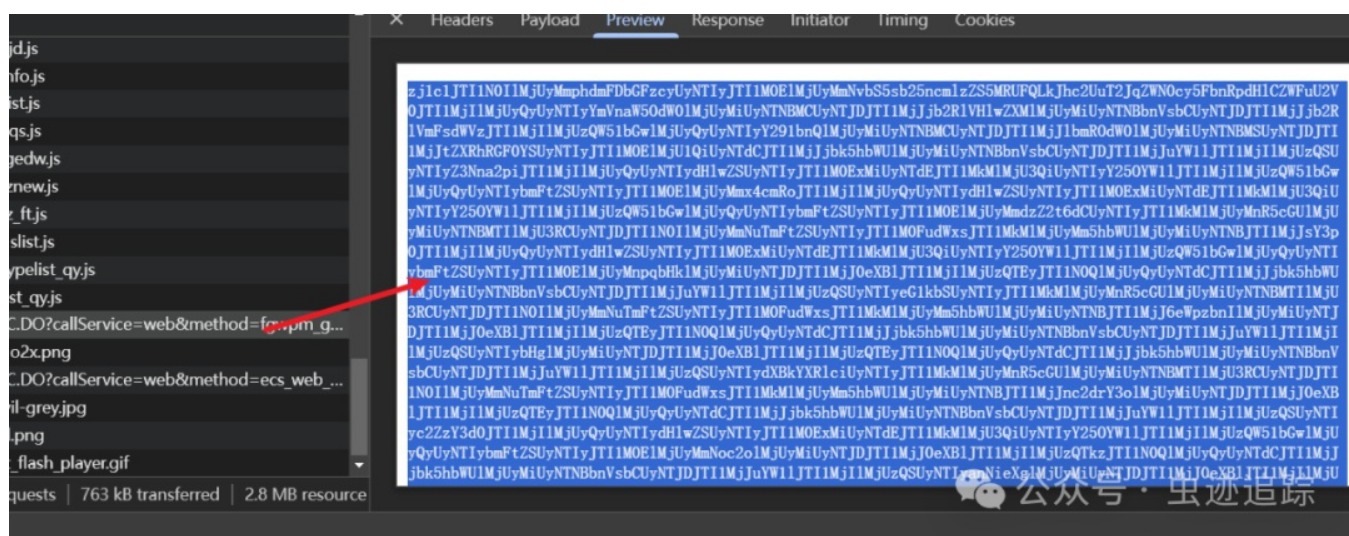
js逆向-人人都会的小栗子

原创 虫迹自踪 虫迹自踪

本文仅供学习参考，如有侵权可私信本人删除，请勿用于其他途径，违者后果自负！如果觉得文章对你有所帮助，可以给博主点击关注和收藏哦！



响应结果是密文，前端展示是明文，很明显是在前端进行了解密。



```
▼Query String Parameters      view source      view URL-encoded
callService: web
method: fgwpm_getJGGSById
sid: 28A85C...E5D92A31E62C8738A0
_website_: *
parlen: 1

▼Form Data      view source      view URL-encoded
a: yqTLmJTI1NUI1MjUyMmNp7N9gBmMzQyYmQ5MDI2YTQ0OGFiMDUxMTB4ZWVwZDc4ZDFh8cPJTI1MjUyMjU1RA==fPbTs
```

```

        h.call(); h = null;
    }
} finally {}
}
};
if (c == null) { c = "a=b$1m1jTI1NU11MjUyMmNdkhzY_mMzQyYmQ5MDI2YTQ0OGFiMDUxMTE4ZWZwZDc4ZDFhO0t9JTl1MjI1MjU1RA==DP00s";
    c = "";
}
G.send(c); G = XMLHttpRequest {onreadystatechange: null, readyState: 1, timeout: 0, withCredentials: false, upload: XMLHttpRequestUp
} else {
    if (c == null) { c = "a=b$1m1jTI1NU11MjUyMmNdkhzY_mMzQyYmQ5MDI2YTQ0OGFiMDUxMTE4ZWZwZDc4ZDFhO0t9JTl1MjI1MjU1RA==DP00s";
        c = "";
    }
    G.send(c);
    XmlHttpRequestHelper.processLID(G);
    if (G.status == 200) {
        if (a != null) {
            var D = G.responseText;
            if (D.toLowerCase().indexOf("text") != -1) {

```

此时发现 **a** 参数已经生成了，网上跟一跟即可发现加密位置，大概在2559-2564行。

```

2553         c = "a" + leapflash.fp.compress(c); c = "a=bS1mJtI1NU1lMJyMnNdkhzY_mMzQyYmQ5MDI2YTQ0OGF1MDUxMTE4ZW50ZDc4ZDZlPh0t9JtI1mJtI1
2554         G.setRequestHeader("Data-Type", "2"); G = XMLHttpRequest {onreadystatechange: null, readyState: 1, timeout: 0, withCreden
2555     }
2556 }
2557 if (lz && b > 0) { z = false, b = 36
2558     if (b > 99) {
2559         c = "a" + Dencode64data(Dbase64encode(pako.Ddeflate(c, { c = "a=bS1mJtI1NU1lMJyMnNdkhzY_mMzQyYmQ5MDI2YTQ0OGF1MDUxMTE4
2560             to: "string"
2561         })));
2562         G.setRequestHeader("Data-Type", "16"); G = XMLHttpRequest {onreadystatechange: null, readyState: 1, timeout: 0, withCreden
2563     } else {
2564         c = "a" + Dencode64data(Dbase64encode(DencodeURIComponent(Descape(c)))); c = "a=bS1mJtI1NU1lMJyMnNdkhzY_mMzQyYmQ5MD
2565         G.setRequestHeader("Data-Type", "14"); G = XMLHttpRequest {onreadystatechange: null, readyState: 1, timeout: 0, withCreden
2566     }
2567 }
2568 } catch (x) {}

```

```

2554         G.setRequestHeader("Data-Type", "2"); G = XMLHttpRequest {onreadystatechange: null, readyState:
2555     }
2556 }
2557 if (!z && b > 0) { z = false, b = 36
2558     if (b > 99) {
2559         c = "a=" + Dencbase64data(Dbase64encode(pako.Ddeflate(c, { c = "\cf342bd9026a448ab051118ea08
2560             to: "string"
2561         })));
2562         G.setRequestHeader("Data-Type", "16"); G = XMLHttpRequest {onreadystatechange: null, readyState:
2563     } else {
2564         c = "a=" + Dencbase64data(Dbase64encode(DencodeURIComponent(Descape(c))));
2565         G.setRequestHeader("Data-Type", "14");
2566     }
2567 } catch (x) {}
2568 G.setRequestHeader("Post-Type", "1");
2569

```

公众号 · 虫迹追踪

详情的位置走了第二个加密，加密函数套了好几层。

```

encbase64data (base64encode (encodeURIComponent (escape (c))));

```

分别步入观察一下，然后将所需要的函数抠下来即可还原。

步骤如下：

```

144 function a(i) {
145     if (i == null || i.length == 0) {
146         return null;
147     }
148     data = [b(5), i, b(5)].join("");
149     var h = Math.floor(data.length * 0.25);
150     var g = Math.floor(data.length * 0.75);
151     var j = c(data);
152     while (j.length < 6) {
153         j += "-";
154     }
155     var f = [data.substring(0, h), j, data.substring(h, g), b(3), data.substring(g)].join("");
156     return f;
157 }
158 if (i.e.hashcode) {

```

encbase64data

公众号 · 虫迹追踪

```

1252 ))(window);
1253
1254 function base64encode(a) {
1255     return btoa(a);
1256 }
1257

```

公众号 · 虫迹追踪

encodeURIComponent和escape为内置函数不用关于关注。

第一分支

继续来看第一个if判断下的加密，基本是一致的，只是使用了不同的加密方式。

```

aHR0cDovLzIwMy45MS40Ni44Mzo4MDMxL0ZHV1BNL3NmZy93c3RiX3Jlc2hvdw==

```

触发该加密也非常简单，首先选择从列表页选择【投资项目公开公示】-【项目建议书公示】再点击详情页即可触发。

```

2555     }
2556 }
2557 if (!z && b > 0) { z = false, b = 255
2558     if (b > 99) {
2559         c = "a=" + Dencbase64data(Dbase64encode(pako.Ddeflate(c, {
2560             to: "string"
2561         })));
2562         G.setRequestHeader("Data-Type", "16");
2563     } else {
2564         c = "a=" + Dencbase64data(Dbase64encode(DencodeURIComponent(Descape(c))));
2565         G.setRequestHeader("Data-Type", "14");
2566     }
2567 } catch (x) {}
2568

```

公众号 · 虫迹追踪

只需要关注 `pako.deflate` 这个函数即可，查询发现这是一个js的第三方库，手动安装执行一下发现结果并不同，网站应该是进行了魔改，可以点进去跟一下逻辑。

```
119
120
121     function b(d, f) { d = "[\\\\"beaname\\\\":\\\\\\"leapwebcluster\\\\",\\\\\\"linkwebsite"
122         var c = new j(f);
123         if (c.push(d, !0), c.err) {
124             throw c.msg || m[c.err];
125         }
126         return c.result;
127     }
128     var p = C("./zlib/deflate"),
129         D = C("./utils/common"),
130         q = C("./utils/strings"),
131         m = C("./zlib/messages"),
132         g = C("./zlib/zstream"),
133         A = Object.prototype.toString,
134         x = 0,
135         B = -1,
136         v = 0,
137         y = 8;
```

`j` 是一个对象，继续进去，在这里可以将 `j` 的所有方法拿下来，然后缺什么补什么也可以得到结果。但是比较麻烦。可以直接全局搜索 `pako` 关键字。

```
8  !function(a) {
9      if ("object" == typeof exports && "undefined" != typeof module) {
10         module.exports = a();
11     } else {
12         if ("function" == typeof define && define.amd) {
13             define([], a);
14         } else {
15             !("undefined" != typeof window ? window : "undefined" != typeof global ? global : "undefined" != typeof self ? self : this).pako = a();
16         }
17     }
18 }
19
20 function a(g, b, h) {
21     function f(m, j) {
22         if (!b[m]) {
23             if (!g[m]) {
24                 var e = "function" == typeof require && require;
25                 if (!j && e) {
26                     return e(m, !0);
27                 }
28                 if (c) {
29                     return c(m, !0);
30                 }
31             }
32         }
33     }
34 }
```

有三处，找到最开头，然后将三目运算的判断精简最后全局导出即可

```
var pako;
!function(a) {
    pako = a()
}(...)
```

类似于这样，就可以了。

验证一下结果

网站:

```
> var c = "11111"
var data = btoa(pako.deflate(c, {
  to: "string"
}));
console.log(data)
eJwzNAQCAALkAPY=
< undefined
>
```

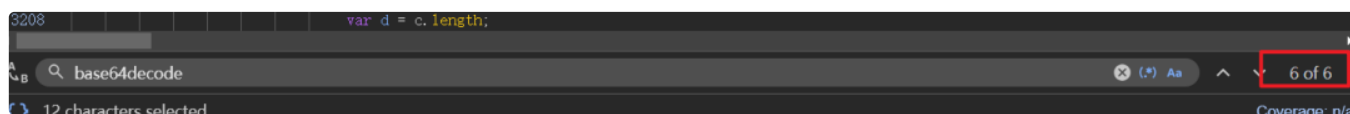
本地:

```
> var c = "11111"
var data = btoa(pako.deflate(c, {
  to: "string"
}));
console.log(data)
eJwzNAQCAALkAPY=
< undefined
>
```

请求体加密到这里就结束了，接下来该到了响应。

响应解密

其实观察加密函数有一个 `base64encode` 那么按照正常逻辑就应该有 `base64decode`，在当前文件中搜一下。



可以将可疑的地方打上断点，然后逐一查看。

```
2694
2695
2696     return D; D = "NukbtJTI1NOI1MjUyMmphaWZDbGFzcyUyNTIyJTI1MOE1MjUyMmNvbS5sb25ncmlzZS5MRUFQLkJhc2
2697   }
2698   } else if (D && G.getResponseHeader('RESPTYPE')) D = "NukbtJTI1NOI1MjUyMmphaWZDbGFzcyUyNTIyJTI1MOE1MjU
2700   ▶ return Dunescape(DdecodeURIComponent(Dbase64decode(Ddecbase64data(D))));
2701
2702   var o = G.getResponseHeader("ServerTime997");
2703   if (o != null && o.length > 0) {
```

`base64decode` -> `atob`，这里同样只需要扣一个函数即可 `decbase64data`。

按部就班没什么难度。

```
function _decbase64data(str) {
  if (str == null || str.length == 0)
    return null;

  if (str.length < 20)
    throw new Error("请求数据长度不合法");

  var len = str.length - 9;
  var pos1 = Math.floor(len * 0.25);
  var pos2 = Math.floor(len * 0.75);
  var hashCode = str.substring(pos1, pos1 + 6);
  var sb = [];
  sb.push(str.substring(0, pos1));
  sb.push(str.substring(pos1 + 6, pos2 + 6));
  sb.push(str.substring(pos2 + 9));

  var lastStr = sb.join('');
  sb.length = 0;
```

+

总结

+

加密比较常规，没有什么特别的地方，很适合练习。

个人观点，仅供参考
